

List of Figures

Figure 4.1	High level overview of Kaltofen Yang algorithm	44
------------	--	----

Chapter 1

Introduction

We know that given n points in $\mathbb{R}^2$ we can fit a polynomial of degree $n - 1$ through it, however, sometimes it is better to fit a rational function over the given points. Rational functions have more flexibility and can model more complex behavior such as, asymptotes, poles, saturation effects, decay rates. This often gives better global accuracy than polynomials [19].

In this work, we will implement the Kaltofen-Yang sparse multivariate rational function interpolation algorithm [14] and use it to solve a parametric system of linear equations $\mathbb{Z}[y_1, \dots, y_m]$. *n parameters $y_1, \dots, y_m$ with integer coefficients.*

Given $d + 1$ points in an n dimensional space $\mathbb{Q}^n$, the objective is to find a sparse multivariate rational function in $\mathbb{Q}(x_1, \dots, x_n)$. Consider the following sparse rational function

Example 1. Suppose we want to interpolate

$$h(x_1, x_2) = \frac{x_1 - x_2}{\frac{4}{5}x_1^3 + \frac{2}{5}x_1x_2 - \frac{3}{4}x_2^2 + 1} \in \mathbb{Q}(x_1, x_2)$$

To interpolate $T(x)$
We construct a modular black box $\mathcal{B}$ for h to avoid expression swell and faster computer arithmetic. The Kaltofen Yang algorithm defines a family of parameterized parallel affine lines $\mathcal{L}_i$ in the ambient space of the rational function we want to interpolate. The Kaltofen Yang algorithm restricts the rational function h on these lines by evaluating its black box $\mathcal{B}$ on random points on the lines $\mathcal{L}_i$. This restriction is a univariate rational function $T(x) \in \mathbb{Z}_p(x)$. We use the maximal quotient rational function reconstruction algorithm to separate the univariate numerators $\hat{f}_i \in \mathbb{Z}_p[x]$ and the denominators $\hat{g}_i \in \mathbb{Z}_p[x]$. We then recover multivariate numerator $f \in \mathbb{Z}_p[x_1, x_2]$ and denominator $g \in \mathbb{Z}_p[x_1, x_2]$ up to a non zero scalar using the Ben-Or Tiwari algorithm. We recover the coefficients using Zippel's transpose Vandermonde solver and perform rational reconstruction using Maple's iratrecon.

of Monagan [1]

We then recover the rational coefficients of h using Chinese remaindering on rational number reconstruction.

1.1 A brief overview of computer arithmetic

There are multiple computational paradigms for performing arithmetic on modern computers. The following are the most widely used

- Fixed point arithmetic.
- Floating point arithmetic, defined by the IEEE 754 standard.
- Multi-precision arithmetic.

Numerical computation uses floating point arithmetic for approximations which are supported by general purpose hardware like CPUs and GPUs.

Computer algebra systems use multi-precision arithmetic paradigm for computing exact values. Multi-precision arithmetic is not supported by general purpose hardware and is implemented through software that is often optimized at assembly level for different processors. Multi-precision arithmetic has no universal standard like IEEE 754, but GNU's GMP (GNU Multiple Precision Arithmetic Library) and MPFR (GNU Multiple Precision Floating-Point Reliable Library) are the most widely used multi-precision libraries and act as functional standards. Multi-precision libraries are the bedrock for computer algebra systems (Maple), cryptography libraries (OpenSSL, libgcrypt), and computational geometry library (CGAL).

1.2 Cost of integer arithmetic operations

Omit ?

We now briefly mention the bit complexity bounds in terms of the number of bits required to represent the operands using the big O notation for some algorithms for performing integer addition and multiplication. They provide information on how the computation cost scales as the size of operands increases. The bit-complexity of two n bit integers for different algorithms for addition and multiplication are as follows.

1.2.1 Addition

Adding two n bit integers has bit complexity of $O(n)$ [16].

1.2.2 Multiplication

The cost of multiplying two integers depends on the chosen multiplication algorithm. Several algorithms with different asymptotic complexities are listed below.

1. The Naïve multiplication algorithm has a bit complexity of $O(n^2)$.

EMP uses ~~a~~ 1, 2, 3, 4, ~~to~~ depending on the lengths of the input integers.

2. Karatsuba multiplication is a divide-and-conquer algorithm that reduces the multiplication of two n -digit integers to three multiplications of half-size numbers with bit complexity $O(n^{\log_2 3}) \approx O(n^{1.585})$ [15].
3. Toom-Cook [17] multiplication generalizes Karatsuba's algorithm. It divides a large integer into smaller parts, uses divide and conquer, and polynomial interpolation to achieve the asymptotic bit complexity of $O(n^{1.465})$.
4. Schönhage-Strassen [21] FFT multiplication uses fast Fourier transforms over modular rings to achieve $O(n \log n \log \log n)$. ~~used by EMP~~
5. Fürer [8] improved the asymptotic bound to $O(n \log n 2^{O(\log^* n)})$.
6. Harvey and van der Hoeven [11] improved the bound to $O(n \log n)$, which is currently the best known asymptotic bound. *and believed to be optimal.*

Division and Gcd. If multiplied $\approx O(mn)$, $\div \approx O(mn)$ and gcd $\approx O(mn \log n)$.

1.3 Performance issue: Intermediate expression swell.

Multi-precision libraries used in computer algebra systems provide fast, optimized implementations to perform arithmetic over $\mathbb{Z}$, $\mathbb{Q}$ and $\mathbb{Z}_p$. Moreover, unlike fixed point arithmetic, which results in overflow when the size of the number exceeds 32 bits in case of single precision and 64 bits in case of double precision, multi-precision libraries do not have an upper bound for the size of the operands and the result. However, this results in a blow up in the size of operands and intermediate results, slowing down computation. Since polynomial interpolation is a linear operator, and we will apply Kaltofen-Yang sparse multivariate rational function interpolation to solve a parametric system of linear equations, it is fitting to consider the following example from [9] to demonstrate expression swell.

Example 2. Solve a system of linear equations with integer coefficients, using Gaussian elimination using only integer arithmetic,

Hadamard
 $|\det(A)| \leq \prod_{i=1}^n \sqrt{\sum_{j=1}^n A_{ij}^2} = 86266$

$$\begin{aligned} 22x + 44y + 74z &= 1, \\ 15x + 14y - 10z &= -2, \\ -25x - 28y + 20z &= 34. \end{aligned}$$

$$A = \begin{bmatrix} 22 & 44 & 74 \\ 15 & 14 & -10 \\ -25 & -28 & 20 \end{bmatrix}, \quad \vec{b} = \begin{bmatrix} 1 \\ -2 \\ 34 \end{bmatrix}$$

$$x_i = \frac{\det(A^i)}{\det A}$$

Gaussian elimination with integers.

$$\left[\begin{array}{ccc|c} 22 & 44 & 74 & 1 \\ 15 & 14 & -10 & -2 \\ -25 & -28 & 20 & 34 \end{array} \right] \xrightarrow{R_2 \leftarrow 22R_2 - 15R_1} \left[\begin{array}{ccc|c} 22 & 44 & 74 & 1 \\ 0 & -352 & -1330 & -59 \\ -25 & -28 & 20 & 34 \end{array} \right]$$

Use column bound $H(A) \leq \det(A) \leq \prod_{j=1}^n \sum_{i=1}^n A_{ij}^2$ and $\|\vec{b}\|_2 = 31$
 $37 \times 54 \times 78 \times 31$
 b and use ~~max~~ $\|Cot$

$$\begin{aligned}
& \left[\begin{array}{ccc|c} 22 & 44 & 74 & 1 \\ 0 & -352 & -1330 & -59 \\ -25 & -28 & 20 & 34 \end{array} \right] \xrightarrow{R_3 \leftarrow -25R_1 + 22R_3} \left[\begin{array}{ccc|c} 22 & 44 & 74 & 1 \\ 0 & -352 & -1330 & -59 \\ 0 & 484 & 2290 & 773 \end{array} \right] \\
& \left[\begin{array}{ccc|c} 22 & 44 & 74 & 1 \\ 0 & -352 & -1330 & -59 \\ 0 & 484 & 2290 & 773 \end{array} \right] \xrightarrow{\begin{array}{l} R_2 \leftarrow -11R_2, \\ R_3 \leftarrow -8R_3, \\ R_3 \leftarrow R_3 + R_2 \end{array}} \left[\begin{array}{ccc|c} 22 & 44 & 74 & 1 \\ 0 & -3872 & -14630 & -649 \\ 0 & 0 & 3690 & 5535 \end{array} \right] \\
& \left[\begin{array}{ccc|c} 22 & 44 & 74 & 1 \\ 0 & -3872 & -14630 & -649 \\ 0 & 0 & 3690 & 5535 \end{array} \right] \xrightarrow{R_2 \leftarrow -3690R_2 + 1330R_3} \left[\begin{array}{ccc|c} 22 & 44 & 74 & 1 \\ 0 & -1298880 & 0 & 7143840 \\ 0 & 0 & 3690 & 5535 \end{array} \right] \\
& \left[\begin{array}{ccc|c} 22 & 44 & 74 & 1 \\ 0 & -1298880 & 0 & 7143840 \\ 0 & 0 & 3690 & 5535 \end{array} \right] \xrightarrow{R_1 \leftarrow -1298880R_1 + 44R_2} \left[\begin{array}{ccc|c} 28575360 & 0 & 96017120 & 315627840 \\ 0 & -1298880 & 0 & 7143840 \\ 0 & 0 & 3690 & 5535 \end{array} \right] \\
& \left[\begin{array}{ccc|c} 28575360 & 0 & 96017120 & 315627840 \\ 0 & -1298880 & 0 & 7143840 \\ 0 & 0 & 3690 & 5535 \end{array} \right] \xrightarrow{R_1 \leftarrow -3690R_1 - 74R_3} \left[\begin{array}{ccc|c} 1257315840 & 0 & 0 & 7543895040 \\ 0 & -1298880 & 0 & 7143840 \\ 0 & 0 & 3690 & 5535 \end{array} \right]
\end{aligned}$$

$$\Rightarrow 1257315840x = 7543895040,$$

$$\Rightarrow -57150720y = 314328960,$$

$$\Rightarrow 162360z = 243540.$$

$$\Rightarrow x = \frac{7543895040}{1257315840} = 6,$$

$$\Rightarrow y = \frac{314328960}{-57150720} = -\frac{11}{2},$$

$$\Rightarrow z = \frac{243540}{162360} = \frac{3}{2}.$$

Even though the final result is small, we end up with some pretty large intermediate values that slow down the computation. ↗

Since the final solution is over $\mathbb{Q}^3$, we can allow fractions while performing row operations to avoid blowing up of the size of intermediate entries.

1. Before eliminating x_k we minimize the growth of the fractions. We call this more Gaussian elimination.

$$x = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} \propto \begin{matrix} \downarrow p \\ \rightarrow \\ \downarrow q \end{matrix} \begin{bmatrix} A \\ B \end{bmatrix} \rightarrow x$$

$$x = \begin{bmatrix} -y_1/2 \\ -y_2/2 \\ -y_3/2 \end{bmatrix} \quad n$$

$\alpha \in \mathbb{Z}_q^m$

Gaussian elimination with fractions.

$$\left[\begin{array}{ccc|c} 22 & 44 & 74 & 1 \\ 15 & 14 & -10 & -2 \\ -25 & -28 & 20 & 34 \end{array} \right] \xrightarrow{R_1 \leftarrow \frac{1}{22} R_1} \left[\begin{array}{ccc|c} 1 & 2 & \frac{37}{11} & \frac{1}{22} \\ 15 & 14 & -10 & -2 \\ -25 & -28 & 20 & 34 \end{array} \right]$$

$$\gcd(37, 11) = \gcd(1, 22) = 1$$

$$\left[\begin{array}{ccc|c} 1 & 2 & \frac{37}{11} & \frac{1}{22} \\ 15 & 14 & -10 & -2 \\ -25 & -28 & 20 & 34 \end{array} \right] \xrightarrow{\begin{array}{l} R_2 \leftarrow R_2 - 15R_1 \\ R_3 \leftarrow R_3 + 25R_1 \end{array}} \left[\begin{array}{ccc|c} 1 & 2 & \frac{37}{11} & \frac{1}{22} \\ 0 & -16 & -\frac{665}{11} & -\frac{59}{22} \\ 0 & 22 & \frac{1145}{11} & \frac{773}{22} \end{array} \right]$$

$$\gcd(665, 11) = \gcd(59, 22), \gcd(1145, 11) = \gcd(773, 22) = 1$$

$$\left[\begin{array}{ccc|c} 1 & 2 & \frac{37}{11} & \frac{1}{22} \\ 0 & -16 & -\frac{665}{11} & -\frac{59}{22} \\ 0 & 22 & \frac{1145}{11} & \frac{773}{22} \end{array} \right] \xrightarrow{R_2 \leftarrow -\frac{1}{16} R_2} \left[\begin{array}{ccc|c} 1 & 2 & \frac{37}{11} & \frac{1}{22} \\ 0 & 1 & \frac{665}{176} & \frac{59}{352} \\ 0 & 22 & \frac{1145}{11} & \frac{773}{22} \end{array} \right]$$

$$\gcd(665, 176) = \gcd(59, 352) = 1.$$

$$\left[\begin{array}{ccc|c} 1 & 2 & \frac{37}{11} & \frac{1}{22} \\ 0 & 1 & \frac{665}{176} & \frac{59}{352} \\ 0 & 22 & \frac{1145}{11} & \frac{773}{22} \end{array} \right] \xrightarrow{\begin{array}{l} R_1 \leftarrow R_1 - 2R_2 \\ R_3 \leftarrow R_3 - 22R_2 \end{array}} \left[\begin{array}{ccc|c} 1 & 0 & -\frac{369}{88} & -\frac{51}{176} \\ 0 & 1 & \frac{665}{176} & \frac{59}{352} \\ 0 & 0 & \frac{1845}{88} & \frac{5535}{176} \end{array} \right]$$

$$\gcd(369, 88) = \gcd(51, 176) = \gcd(1845, 88) = \gcd(5535, 176) = 1.$$

$$\left[\begin{array}{ccc|c} 1 & 0 & -\frac{369}{88} & -\frac{51}{176} \\ 0 & 1 & \frac{665}{176} & \frac{59}{352} \\ 0 & 0 & \frac{1845}{88} & \frac{5535}{176} \end{array} \right] \xrightarrow{R_3 \leftarrow \frac{88}{1845} R_3} \left[\begin{array}{ccc|c} 1 & 0 & -\frac{369}{88} & -\frac{51}{176} \\ 0 & 1 & \frac{665}{176} & \frac{59}{352} \\ 0 & 0 & 1 & \frac{487080}{324720} \end{array} \right]$$

$$\gcd(487080, 324720) = 162360.$$

Normalization of the fraction:

$$\frac{487080 \frac{1}{162360}}{324720 \frac{1}{162360}} = \frac{3}{2}$$

$$\left[\begin{array}{ccc|c} 1 & 0 & -\frac{369}{88} & -\frac{51}{176} \\ 0 & 1 & \frac{665}{176} & \frac{59}{352} \\ 0 & 0 & 1 & \frac{3}{2} \end{array} \right] \xrightarrow{R_2 \leftarrow R_2 - \frac{665}{176} R_3} \left[\begin{array}{ccc|c} 1 & 0 & -\frac{369}{88} & -\frac{51}{176} \\ 0 & 1 & 0 & -\frac{1936}{352} \\ 0 & 0 & 1 & \frac{3}{2} \end{array} \right]$$

$$\gcd(1936, 352) = 176.$$

$$\left[\begin{array}{ccc|c} 1 & 0 & -\frac{369}{88} & -\frac{51}{176} \\ 0 & 1 & 0 & -\frac{11}{2} \\ 0 & 0 & 1 & \frac{3}{2} \end{array} \right] \xrightarrow{R_1 \leftarrow R_1 + \frac{369}{88} R_3} \left[\begin{array}{ccc|c} 1 & 0 & 0 & \frac{1056}{176} \\ 0 & 1 & 0 & -\frac{11}{2} \\ 0 & 0 & 1 & \frac{3}{2} \end{array} \right] \neq 6$$

$$\gcd(1056, 176) = 176$$

Normalization of the fraction:

$$\frac{1056 \frac{1}{176}}{176 \frac{1}{176}} = 6$$

Thus, we recovered the solution of the system

$$x = 6, \quad y = -\frac{11}{2}, \quad z = \frac{3}{2}.$$

$$\frac{a}{b} + \frac{c}{d} = \frac{ad + bc}{bcd} \text{ gcd.}$$

which takes 3 x ad and bc +
one gcd and two divisions

We see that performing arithmetic over rational numbers can also lead to a larger expression as we get large fractions like $\frac{487080}{324720}$ in the matrix. Moreover, addition and multiplication operations on the rationals are slow. For adding two fractions, we first find the common denominator that takes one multiplication, scale the numerators, that is, two more multiplications, then there is an addition, a gcd computation and if the $\gcd \neq 1$, two divisions to get the fraction in the simplest form. Similarly, for fraction multiplication, there are two multiplications (for the numerator and denominator), a gcd computation, and if the $\gcd \neq 1$, there are two divisions to get the fraction in the simplest form. So, using rationals is also inefficient. We therefore use modular arithmetic in a finite field $\mathbb{Z}_p$. It ~~solves~~ has two advantages.

MODULAR ALGORITHM.

1. The elements of the field $\mathbb{Z}_p$ are less than p so that bounds the size of the operands from above.

$$a \cdot b = (ab) \bmod p.$$

2. Performing additions and multiplications on $\mathbb{Z}_p$ does not require as many computations as performing additions and multiplications on $\mathbb{Q}$.

~~Does require more but~~

After doing all the arithmetic, we can use the Chinese remainder theorem with the extended Euclidean algorithm to recover the rational pre-images from the modular images of the result.

Solution: 6, -11/2, 3/2

1.3.1 Rational reconstruction bound

If we want to use a modular field to do all the intermediate computations and recover the rational pre-image from the modular result, an important question to ask is what prime to use. Wang [22] came up with the following bound that helps us select a prime. Suppose that we want to recover a reduced rational number

how many primes -

the primes -

$$\frac{n}{d} \in \mathbb{Q}, \text{ such that } \gcd(n, d) = 1, \quad d > 0,$$

from its modular image

$$a \equiv nd^{-1} \pmod{p},$$

mod m = ~~1000~~

where p is a prime modulus, then

$$mp > 2B^2.$$

m = $\prod p_i$ and

where,

$$B = \max(|n|, |d|)$$

This bound guaranties that among all rational numbers, the numerator and denominator do not exceed B and can uniquely recover the original fraction from a single modular evaluation. For the example in question, we have already computed its solution on $\mathbb{Z}$ and $\mathbb{Q}$, so we know the solutions to find the smallest prime needed to compute the recover the rational pre-images, however, in practice we use large machine primes like $2^{31} - 1$ for modular arithmetic.

$$x = 6, \quad y = -\frac{11}{2}, \quad z = \frac{3}{2}.$$

For the rational number

$$\frac{n}{d} = -\frac{11}{2},$$

we have

$$|n| = 11, \quad |d| = 2.$$

Thus

$$B = \max(|n|, |d|) = 11.$$

To guarantee unique rational reconstruction, we require

$$p > 2B^2 = 2 \cdot 11^2 = 242.$$

Hence any prime $p > 242$ suffices. The smallest such prime is $p = 251$.
is the smallest prime. But we not ended the size of the solutions in advance!
allowing for +ve and -ve numerators
 $p = 251$.

Modular Gaussian elimination over $\mathbb{Z}_p$, where $p = 251$.

The augmented matrix over $\mathbb{Z}_{251}$ is

$$\left[\begin{array}{ccc|c} 22 & 44 & 74 & 1 \\ 15 & 14 & 241 & 249 \\ 226 & 223 & 20 & 34 \end{array} \right],$$

since $-10 \equiv 241$, $-2 \equiv 249$, $-25 \equiv 226$, and $-28 \equiv 223 \pmod{251}$.

$$22^{-1} \equiv 194 \pmod{251}$$

$$\left[\begin{array}{ccc|c} 22 & 44 & 74 & 1 \\ 15 & 14 & 241 & 249 \\ 226 & 223 & 20 & 34 \end{array} \right] \xrightarrow{R_1 \leftarrow 194 R_1} \left[\begin{array}{ccc|c} 1 & 2 & 49 & 194 \\ 15 & 14 & 241 & 249 \\ 226 & 223 & 20 & 34 \end{array} \right].$$

$$\left[\begin{array}{ccc|c} 1 & 2 & 49 & 194 \\ 15 & 14 & 241 & 249 \\ 226 & 223 & 20 & 34 \end{array} \right] \xrightarrow{\substack{R_2 \leftarrow R_2 - 15R_1 \\ R_3 \leftarrow R_3 - 226R_1}} \left[\begin{array}{ccc|c} 1 & 2 & 49 & 194 \\ 0 & 235 & 8 & 100 \\ 0 & 22 & 241 & 115 \end{array} \right].$$

$$235^{-1} \equiv 47 \pmod{251}$$

$$\left[\begin{array}{ccc|c} 1 & 2 & 49 & 194 \\ 0 & 235 & 8 & 100 \\ 0 & 22 & 241 & 115 \end{array} \right] \xrightarrow{R_2 \leftarrow 47 R_2} \left[\begin{array}{ccc|c} 1 & 2 & 49 & 194 \\ 0 & 1 & 125 & 182 \\ 0 & 22 & 241 & 115 \end{array} \right].$$

$$\left[\begin{array}{ccc|c} 1 & 2 & 49 & 194 \\ 0 & 1 & 125 & 182 \\ 0 & 22 & 241 & 115 \end{array} \right] \xrightarrow{\substack{R_1 \leftarrow R_1 - 2R_2 \\ R_3 \leftarrow R_3 - 22R_2}} \left[\begin{array}{ccc|c} 1 & 0 & 50 & 81 \\ 0 & 1 & 125 & 182 \\ 0 & 0 & 1 & 127 \end{array} \right].$$

$$\left[\begin{array}{ccc|c} 1 & 0 & 50 & 81 \\ 0 & 1 & 125 & 182 \\ 0 & 0 & 1 & 127 \end{array} \right] \xrightarrow{\substack{R_1 \leftarrow R_1 - 50R_3 \\ R_2 \leftarrow R_2 - 125R_3}} \left[\begin{array}{ccc|c} 1 & 0 & 0 & 6 \\ 0 & 1 & 0 & 120 \\ 0 & 0 & 1 & 127 \end{array} \right].$$

Thus, over $\mathbb{Z}_{251}$, the solution is

$$x \equiv 6, \quad y \equiv 120, \quad z \equiv 127 \pmod{251}.$$

NB: This requires $\det(A) \neq 0$.

Maple's `iratrecon` method implements Wang's algorithm, using which we can recover the rational images.

$$\text{iratrecon}(120, 251) = \frac{-11}{2}, \quad \text{iratrecon}(127, 251) = \frac{3}{2}.$$

How do you bound the solutions given A, b ?

1.4 Black boxes

In computer algebra, the "black box model" refers primarily to an implicit computational model for an arbitrary element over some algebraic object [13]. A black box is implemented as a procedure in a computer algebra system that accepts appropriate input from the user to pick a particular element from the underlying algebraic object, perform the appropriate computations, and return the output, if one exists or return a fail message. Consider the following examples over some commutative ring R and field $\mathbb{F}$ that can be represented as a black box which returns,

- the value of $f(\alpha_1, \dots, \alpha_n) \in R$ or $\mathbb{F}$ for some polynomial $f(x_1, \dots, x_n) \in R[x_1, \dots, x_n]$ or $\mathbb{F}[x_1, \dots, x_n]$ at some point $\alpha \in R^n$ or $\mathbb{F}^n$.
- the value of $\frac{f(\alpha_1, \dots, \alpha_n)}{g(\alpha_1, \dots, \alpha_n)} \in \mathbb{F}$ for some rational function $\frac{f(x_1, \dots, x_n)}{g(x_1, \dots, x_n)}$ over a field of rational functions $\mathbb{F}(x_1, \dots, x_n)$.

- the determinant $\text{Det}(A) \in R[y_1, \dots, y_m]$ or $\mathbb{F}[y_1, \dots, y_m]$ of some matrix $A \in M_n(R[y_1, \dots, y_m])$, or $M_n(\mathbb{F}[y_1, \dots, y_m])$.

$$x \in R(y_1, \dots, y_n) \text{ or } \mathbb{F}(y_1, \dots, y_n)$$

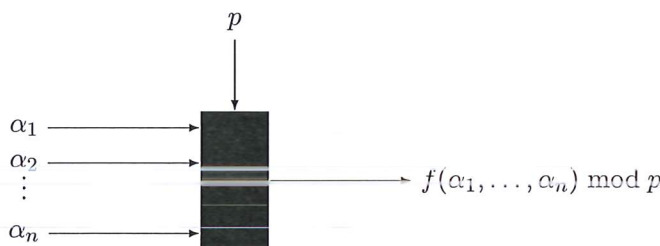
- the solution vector x for some parametric system of linear equations $Ax = b$, where $A \in M_n(R[y_1, \dots, y_m])$, or $M_n(\mathbb{F}[y_1, \dots, y_m])$ and $b \in R^n$ or $\mathbb{F}^n$.

The black box $\mathcal{B}$ of an algebraic object hides its internal structure and complexity and emulates certain underlying mathematical properties, like the linearity property in the case of polynomial black boxes. Probing or querying the black box of an object produces its image under the evaluation homomorphism.

1.5 Polynomial black boxes and evaluation homomorphisms

1.5.1 Modular black box for a multivariate polynomial

Definition 1.5.1. Let f be a polynomial in n variables $x_1, \dots, x_n$ over a field $\mathbb{F}$ ie. $f \in \mathbb{F}[x_1, \dots, x_n]$. A modular black box $\mathcal{B}$ for f is a program that on input $\alpha \in \mathbb{F}^n$ and a prime p computes, $f(\alpha) \bmod p$ ie. $\mathcal{B} : \mathbb{F}^n \rightarrow \mathbb{Z}_p$.



The following theorem from Dummit and Foote

Theorem 1.5.1. [6] Let $\mathbb{F}[x]$ be a ring of polynomials over a field $\mathbb{F}$ and let $I \subset \mathbb{F}[x]$ be an ideal of the ring, then $\mathbb{F}[x]/I$ is a field if and only if I is a maximal ideal.

This theorem also holds for the multivariate polynomial ring $\mathbb{F}[x_1, \dots, x_n]$.

Definition 1.5.2. Let $\mathbb{F}$ be a field and $\mathbb{F}[x_1, \dots, x_n]$ be the ring of polynomials with n variables. Let $f(x_1, \dots, x_n) \in \mathbb{F}[x_1, \dots, x_n]$ and $\alpha \in \mathbb{F}^n$ be a point in the field. The evaluation homomorphism is a map

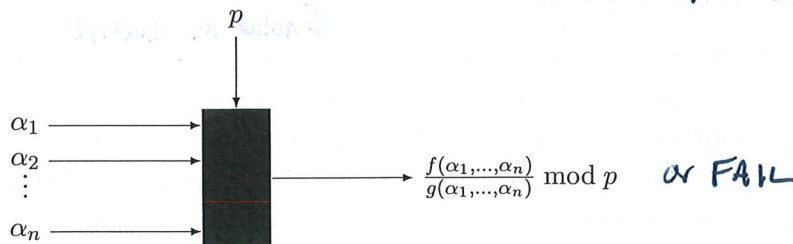
$$\begin{aligned} \phi : \mathbb{F}[x_1, \dots, x_n] &\rightarrow \mathbb{F}[x_1, \dots, x_n]/(x_1 - \alpha_1, \dots, x_n - \alpha_n) \cong \mathbb{F} \\ f(x_1, \dots, x_n) &\mapsto f(\alpha_1, \dots, \alpha_n) \end{aligned}$$

Let $\mathcal{B}$ be the black box for $f(x) \in \mathbb{F}[x]$. Probing or querying $\mathcal{B}$ at α is the same as taking the image under the evaluation homomorphism ϕ .

1.5.2 Modular black boxes for rational functions

Definition 1.5.3. Let $\frac{f}{g}$ be a rational polynomial in n variables $x_1, \dots, x_n$ over a field of rational functions $\mathbb{F}$ ie. $\frac{f}{g} \in \mathbb{F}(x_1, \dots, x_n)$. A modular black box $\mathcal{B}$ for $\frac{f}{g}$ is a program that on input $\alpha \in \mathbb{F}^n$ and a prime p computes, $\frac{f(\alpha)}{g(\alpha)} \mod p$ ie. $\mathcal{B} : \mathbb{F}^n \rightarrow \mathbb{Z}_p$.

If $g(\alpha) = 0$
it outputs FAIL.



Theorem 1.5.2. Evaluating a modular black for a multivariate polynomial $f \in \mathbb{F}[x_1, \dots, x_n]$ at t distinct points will produce a vector in $\mathbb{F}_p^t$.

Proof. Let $\mathcal{B}$ be a modular black box, let $\vec{\alpha}_i = \begin{bmatrix} \alpha_{i1} \\ \vdots \\ \alpha_{in} \end{bmatrix} \in \mathbb{F}^n$, where $1 \leq i \leq T$ be the points

at which we evaluate $\mathcal{B}$ for some prime p . We know that each point $\vec{\alpha}_i$ corresponds to the maximal ideal $\mathcal{I}_i = (x_1 - \alpha_{i1}, \dots, x_n - \alpha_{in}) \subset \mathbb{F}[x_1, \dots, x_n]$.

From the Chinese remainder theorem [6] we get that the map

$$\phi : \mathbb{F}[x_1, \dots, x_n] \rightarrow \prod_{i=1}^t \mathbb{F}[x_1, \dots, x_n] / \mathcal{I}_i$$

is a ring homomorphism with kernel $\bigcap_{i=1}^t \mathcal{I}_i$. If the ideals are distinct then $\bigcap_{i=1}^t \mathcal{I}_i = \prod_{i=1}^t \mathcal{I}_i$ and the map ϕ is surjective. Moreover,

$$\Rightarrow \prod_{i=1}^t \mathbb{F}[x_1, \dots, x_n] / \mathcal{I}_i \cong \mathbb{F}[x_1, \dots, x_n] / \prod_{i=1}^t \mathcal{I}_i \cong \mathbb{F}^t. \quad (1.1)$$

Evaluating the modular black box $\mathcal{B}$ at multiple points give us $\prod_{i=1}^t \mathbb{F}[x_1, \dots, x_n] / \mathcal{I}_i$, which from 1.1 $\cong \mathbb{F}^t$. We can think of modular black box as a composition of modular homomorphism after evaluation homomorphism, thus giving us $\mathbb{F}^t \rightarrow \mathbb{F}_p^t$. $\square$

1.6 Randomized algorithm, early termination

1.6.1 Randomized algorithms

Although using modular black boxes reduces the cost of doing arithmetic, they introduce another challenge. What if the point with which we query the black box sends the result

Only true of rational functions and rational numbers or $f(x)=0$ or $\frac{a}{b} \in \mathbb{Q}$ ad p/b.

Include an example.

of the computation to the kernel of the modular homomorphism. This will generate a zero, leading to loss of information and wastage of computations, effectively failing to produce a result. ~~This introduces randomization~~ Since we do not know before hand which points will result in failure, we must query the black box with randomly sampled points.

1.6.2 Monte Carlo algorithms

Include an example

Monte Carlo algorithms use repeated random sampling to produce output in finite time, however, they have non zero failure probability.

For an algorithm using modular black boxes, failure occurs when one of the sampled points finds a result that is in the kernel of the modular homomorphism. Whenever the algorithm returns a zero, we must choose a different point and rerun the algorithm.

1.6.3 Failure probability

If we have a modular black box for a non-zero polynomial over a finite field $\mathbb{F}_p$, then the kernel of the evaluation homomorphism will be the roots of the polynomial, and its failure probability is given by the Schwartz-Zippel lemma.

Lemma 1.6.1 (Schwartz Zippel lemma). Let $f(x_1, \dots, x_n) \in \mathbb{F}_p[x_1, \dots, x_n]$ be such that $f \neq 0$ and $S \subset \mathbb{F}_p$ be large and finite. If we randomly sample ~~points~~ α from S^n then

$$\Pr[\text{failure}] = \Pr[f(\alpha) = 0] = \frac{\text{degree}(f)}{|S|}$$

If we choose a large prime p , and let S be equal to $\mathbb{F}_p$, we can make the failure probability as low as we desire, thus making sure that the algorithm gives the correct output with high probability (whp).

We can use two points α_1, α_2 to reduce the probability of failure.

1.6.4 Early termination

While using black boxes for polynomial interpolation, we loose information about the polynomials like ~~its~~ ^{their} degree and the number of terms ^{they} it contains. This makes it challenging to come up with a bound that acts as a termination condition beforehand for an iterative algorithm using the black box.

Numerical algorithms use the notion of convergence of error as a termination condition. We need something analogous for algorithms that use black boxes for polynomials.

Early termination is a probabilistic technique in black box sparse polynomial interpolation that allows algorithms to adaptively detect and terminate once the polynomial is interpolated, without requiring any information beforehand regarding the degree and the number of monomials of the polynomial. This dramatically reduces black box probes to the black box.

Often we do not know

No one knows how to bound $\#f$ or $\#$ terms of f .

Let $f \in \mathbb{F}_p[x]$

Suppose we randomly pick $\alpha_0, \alpha_1, \alpha_2, \dots$ from S and compute $y_i = f(\alpha_i)$. Let $\hat{f}_i(x)$ be the interpolating polynomial for the points (α_k, y_k) for $0 \leq k \leq i$. When $\hat{f}_i(\alpha) = \hat{f}_{i+1}(\alpha)$, $\hat{f}_i = \hat{f}_{i+1}$ with high probability.

Let $f(x) \in \mathbb{F}_p[x]$ and $S \subset \mathbb{F}_p$ be large and finite. We randomly sample points s_i from S for polynomial interpolation. Once the correct interpolating polynomial $\hat{f}$ has been reconstructed, future probes satisfy $f(\alpha) = \hat{f}(\alpha)$ whp, probability. Thus, we can terminate the algorithm once the output stabilizes. interpolating polynomial $\hat{f}$ stabilizes.

1.7 Sparse polynomials

Suppose $f \in \mathbb{F}[x_1, \dots, x_n]$ and $\deg(f, x_i) = d_i$ and $T(f) = T(d_i + 1)$.

Polynomial interpolation algorithms can be classified into two categories on the basis of how their performance scales. Algorithms like multivariate Lagrange interpolation, multivariate Newton divided differences, and Kronecker substitution-based methods scale with the total degree of the polynomial [9], while algorithms like Ben-Or Tiwari and Zippel's algorithm scale on number of nonzero terms. which can be much less than D .

1.7.1 Sparse polynomial definition and example

Definition 1.1. Let f be a non-zero polynomial in $x_1, x_2, \dots, x_n$. Let t denote the number of non-zero terms in f such that $t \geq 1$ and let $d = \deg(f)$ be the total degree of f . The maximum number of possible terms in f is

$$M = \binom{n+d}{d}.$$

We say that f is *sparse* if $t < \sqrt{M}$.

Example 1.2. Let $f = x_1 x_2^3 x_5 + 2x_1 x_3 x_4$. Here $t = 2$, $d = 5$, $n = 5$, and

$$M = \binom{5+5}{5} = \binom{10}{5} = 252.$$

$$T(f) = 2 \cdot 4 \cdot 2 \cdot 2 \cdot 2 = 64.$$

Therefore, f is a sparse polynomial since

$$t = 2 < \sqrt{M} = \sqrt{252} \approx 15.9.$$

Definition 1.3. A sparse polynomial f is normally represented as

$$f = \sum_{k=1}^t c_k x_1^{e_{k,1}} x_2^{e_{k,2}} \dots x_n^{e_{k,n}}, \quad \text{where } c_k \neq 0.$$

Definition 1.4. A multivariate rational function f/g with $\gcd(f, g) = 1$ is *sparse* if both polynomials f and g are sparse.

Let $f = \sum_{i=1}^t a_i \prod_{j=1}^n x_j^{e_{ij}} = M_i(x_1, \dots, x_n) \in \mathbb{Z}[x_1, \dots, x_n]$.
 31 bit primes $\equiv$
 Reference.

1.7.2 Sparse multivariate polynomial interpolation Ben-Or Tiwari

Ben-Or Tiwari is a two-step multivariate interpolation algorithm that interpolates all the variables of the polynomial simultaneously as opposed to Zippel's multivariate interpolation algorithm, which interpolates the variables one at a time.

For efficiency, we will use Ben-Or Tiwari algorithm to interpolate $f(x_1, \dots, x_n) \in \mathbb{Q}[x_1, \dots, x_n]$ modulo a prime p . We are using $p = 2^{31} - 1$ for our computations.

Let B be the modular black box of $f(x_1, \dots, x_n) \in \mathbb{Q}[x_1, \dots, x_n]$.

We evaluate the black box B at the point $[2^i, 3^i, 5^i, \dots, p_n^i]^T \in \mathbb{Z}_p^n$, where p_n is the n^{th} prime.

This gives us $f(2^i, 3^i, 5^i, \dots, p_n^i) \mod p$, for $0 \leq i \leq 2t$, where t is the number of non zero monomials of f .

We will show in chapter 3 that $\{f(2^i, 3^i, 5^i, \dots, p_n^i)\}$ forms a linear recursive sequence of order t . In the first step of the Ben-Or Tiwari algorithm, we use the Berlekamp Massey Euclidean algorithm (BMEA) to find the minimal characteristic polynomial $\Lambda(Z)$ for the linear recursive sequence $\{f(2^i, 3^i, 5^i, \dots, p_n^i)\}$. We factorize $\Lambda(z)$ to find its roots r_i , where

$$r_i = M_i(2, 3, 5, \dots, p_n) \quad (1.2)$$

For this to work for each monomial $M_i(x_1, \dots, x_n)$ of $f(x_1, \dots, x_n)$ we need $M_i(2, 3, 5, \dots, p_n) < p$. This is a significant restriction in practice.

If we determine the total degree $D = \deg(f)$, then $M_i(2, 3, 5, \dots, p_n) < p_n^D$.

We find the monomials $M_i(x_1, \dots, x_n)$ of f by trial division. The second step of the Ben-Or Tiwari algorithm uses Zippel's Vandermonde transpose solver to find the coefficients of the polynomial f . The first step uses insights from linear recursive sequences and uses the Berlekamp Massey algorithm to recover the monomials, while the second step uses Zippel's transpose Vandermonde solver to recover the coefficients.

Finally, we perform Maple's *ratrecon* to recover the rational pre-images of the modular coefficients we computed.

1.7.3 Sparse multivariate rational function interpolation using Kaltofen Yang

Let $h(x_1, \dots, x_n) = \frac{f(x_1, \dots, x_n)}{g(x_1, \dots, x_n)} \in \mathbb{Q}(x_1, \dots, x_n)$, be a multivariate sparse rational function that we want to interpolate, B be the modular black box for $h = \frac{f}{g}$. The Kaltofen Yang

algorithm returns $\frac{f(x_1, \dots, x_n)}{c}$ and $\frac{g(x_1, \dots, x_n)}{c} \in \mathbb{Q}[x_1, \dots, x_n]$ for some scalar $c \in \mathbb{Q}$ as output.

Next we solve for the unknown coefficients a_i .

We have $f(2^i, 3^i, \dots, p_n^i) = \sum_{j=1}^t a_j M_j$ for $0 \leq i < t$,
 a Vandermonde linear system.

Provided $M_i \neq M_j$ it has a unique solution.

Requires $p > M_i$ ensures $M_i \neq M_j$. 13

Suppose we want to compute $f(x_1, \dots, x_n)$ and $g(x_1, \dots, x_n)$ at $\sigma^i \in \mathbb{Z}_p^n$ for $i = 0, 1, 2, \dots$.

In Chapter 4 we will see that the Kaltofen–Yang algorithm constructs a family of parallel affine lines $\mathcal{L}_i$ by the parameterized morphism

$$\begin{aligned} \Psi_{\sigma^i, \beta} : \mathbb{A}_{\mathbb{Z}_p}^1 &\longrightarrow \mathbb{A}_{\mathbb{Z}_p}^n \\ x &\longmapsto (x, \beta_2(x - \sigma_1^i) + \sigma_2^i, \dots, \beta_n(x - \sigma_1^i) + \sigma_n^i). \end{aligned} \quad (\sigma^i) = (\sigma_1^i, \sigma_2^i, \dots, \sigma_n^i)$$

Here, β specifies the direction of the line, and together with σ^i it also specifies the point through which the line passes. We choose β randomly, once at the beginning of the algorithm, and use the same β throughout. We only change σ^i , thus selecting a new line in the direction specified by β passing through a different point.

Why?

to fix the scalar.

$$\Psi_{\sigma^i, \beta}(x) = \begin{bmatrix} 0 \\ \sigma_2^i - \beta_2 \sigma_1^i \\ \vdots \\ \sigma_n^i - \beta_n \sigma_1^i \end{bmatrix} + x \begin{bmatrix} 1 \\ \beta_2 \\ \vdots \\ \beta_n \end{bmatrix}. \quad (1.3)$$

We restrict $\Psi_{\sigma^i, \beta}(x)$ to its image $\text{Im}(\Psi_{\sigma^i, \beta}(x)) = \mathcal{L}_i \subset \mathbb{A}_{\mathbb{Z}_p}^n$, as

$$\bar{\Psi}_{\sigma^i, \beta} : \mathbb{A}_{\mathbb{Z}_p}^1 \longrightarrow \mathcal{L}_i \quad (1.4)$$

Let $h = f/g$ be an unknown multivariate rational function that we want to interpolate. We define the restriction of h to the affine line $\mathcal{L}_i$ as

$$h|_{\mathcal{L}_i} = \left(\frac{f}{g} \right) \circ \bar{\Psi}_{\sigma^i, \beta}(x) = \frac{f(\bar{\Psi}_{\sigma^i, \beta}(x))}{g(\bar{\Psi}_{\sigma^i, \beta}(x))} \in \mathbb{Z}_p(x).$$

This restriction gives a univariate rational function in the function field $\mathbb{Z}_p(x)$.

By evaluating the modular black box $\mathcal{B}$ of h at a point $\mathcal{L}_i(\alpha_j)$ we compute

$$h(\mathcal{L}_i(\alpha_j)) = \frac{f(\bar{\Psi}_{\sigma^i, \beta}(\alpha_j))}{g(\bar{\Psi}_{\sigma^i, \beta}(\alpha_j))} \in \mathbb{Z}_p.$$

Since the restriction of h to the affine line $\mathcal{L}_i$ is a univariate rational function, we can recover $\hat{f}_i(x)$ and $\hat{g}_i(x) \in \mathbb{Z}_p[x]$ from $f(\bar{\Psi}_{\sigma^i, \beta}(\alpha_j))$ and $g(\bar{\Psi}_{\sigma^i, \beta}(\alpha_j))$ up to a non-zero scalar using a rational function reconstruction method such that

The scalar

$$\hat{f}_i(x) = \frac{f(\bar{\Psi}_{\sigma^i, \beta}(x))}{c} \quad \text{and} \quad \hat{g}_i(x) = \frac{g(\bar{\Psi}_{\sigma^i, \beta}(x))}{c}$$

We are using the maximal quotient rational function reconstruction (MQRFR) algorithm for this step. The number of points $\mathcal{L}_i(\alpha_j)$ on the affine line required to successfully recover

$\hat{f}_i(x)$ and $\hat{g}_i(x) = \deg(\hat{f}_i) + \deg(\hat{g}_i) + 2$. ~~The extra point~~ Since we do not know $\deg(\hat{f}_i)$ and $\deg(\hat{g}_i)$ in advance, we need at least one extra point $\mathbb{Z}$ needed to determine the degrees w.h.p.

from $\check{f}_i(\sigma^i) = f(\sigma^i)$ and $\check{g}_i(\sigma^i) = g(\sigma^i)$

We then use the Ben-Or Tiwari algorithm to interpolate the numerator $f(x_1, \dots, x_n)$ and the denominator $g(x_1, \dots, x_n)$ from $\hat{f}_i(x)$ and $\hat{g}_i(x)$, respectively.

We use the same setup for the Ben-Or Tiwari algorithm as described in subsection 1.7.2. Our algorithm implementation expects $f(2^i, 3^i, 5^i, \dots, p_n^i) \bmod p$, and $g(2^i, 3^i, 5^i, \dots, p_n^i) \bmod p$, where $0 \leq i \leq 2 \max(t_{\text{num}}, t_{\text{den}})$ to recover monomials of the numerator and denominator as $\mathcal{M}_{\text{num } i}(2, 3, 5, \dots, p_n)$ and $\mathcal{M}_{\text{den } i}(2, 3, 5, \dots, p_n)$ respectively.

We achieve this using $\sigma^i = [2^i, 3^i, 5^i, \dots, p_n^i]$ to define our parametric affine lines $\mathcal{L}_i$ as

$$\overline{\Psi}_{\sigma^i, \beta}(x) = \begin{bmatrix} 0 \\ 3^i - 2^i \beta_2 \\ \vdots \\ p_n^i - 2^i \beta_n \end{bmatrix} + x \begin{bmatrix} 1 \\ \beta_2 \\ \vdots \\ \beta_n \end{bmatrix}. \quad (1.5)$$

We then recover the coefficients of the numerator and denominator using Zippel's transpose Vandermonde solver and then recover the coefficients using Maple's *iratrecon*.

from [] reference

1.7.4 Application: Solving parametric linear system of equations $Ax = b$.

Consider the parametric linear system $Ax = b$ where

$$A \in \mathbb{Z}[y_1, y_2, \dots, y_m]^{n \times n}$$

is the coefficient matrix of full rank n and

$$b \in \mathbb{Z}[y_1, y_2, \dots, y_m]^n.$$

The variables $y_1, \dots, y_m$ are parameters.

The solution vector $x \in \mathbb{Z}(y_1, \dots, y_m)^n$.

$$x = \begin{bmatrix} x_1 & x_2 & \dots & x_n \end{bmatrix}^T = \begin{bmatrix} \frac{f_1}{g_1} & \frac{f_2}{g_2} & \dots & \frac{f_n}{g_n} \end{bmatrix}^T$$

such that for $f_i, g_i \in \mathbb{Z}[y_1, y_2, \dots, y_m]$, $g_i \neq 0$, $g_i \mid \det(A)$, and $\gcd(f_i, g_i) = 1$ for $1 \leq i \leq n$.

~~Cramer's rule~~

From Cramer's rule, we know that the solutions of $Ax = b$ are given by

$$x_i = \frac{\det(A_i)}{\det(A)} \in \mathbb{Z}(y_1, \dots, y_m) \implies \det(A_i) = x_i \det(A)$$

where A_i is the matrix obtained by replacing the i -th column of A with b , and $\det(A) \in \mathbb{Z}[y_1, y_2, \dots, y_m]$.

$$\frac{f_i(\sigma_i)}{g_i(\sigma_i)}$$

$$\rightarrow T_i(x) = \frac{f(\phi)}{g(\phi)} = \frac{f_i(x)}{g_i(x)}$$

$$T_i(\sigma_i) = \frac{f(\sigma_i)/c}{g(\sigma_i)/c} \rightarrow \text{nt. f}$$

Bareiss/Edmonds algorithm

The Bareiss/Edmonds fraction-free Gaussian elimination algorithm [1, 7], triangularizes an augmented matrix $B = [A \mid b]$ to obtain $\det(A) \in \mathbb{Z}[y_1, y_2, \dots, y_m]$, and then solves the linear system by back substitution.

Expression swell in Bareiss/Edmonds algorithm

Lipson

References.

The Bareiss/Edmonds algorithm computes the leading principal minors $B_{k,k}$ (determinant of the principal $k \times k$ submatrix of A). However, in the last step the determinant of coefficient matrix A is computed.

$$B_{n,n} = \frac{B_{n-1,n-1}B_{n,n} - B_{n,n-1}B_{n-1,n}}{B_{n-2,n-2}} = \det(A) \quad (1.6)$$

the numerator in equation 1.6 is the product of the determinants $B_{n,n}$ and $B_{n-2,n-2}$, which is a huge polynomial in $\mathbb{Z}[y_1, y_2, \dots, y_m]$ that exceeds both in degree and number of monomials.

can be much larger than $\det(A)$.

Need some data

Solving system of linear equations using Kaltofen Yang sparse multivariate rational function interpolation.

Let $Ax = b$ be a linear system such that $A_{n \times n}$ and $\forall a_{ij} \in A, a_{ij} \in \mathbb{F}[y_1, \dots, y_m]$, then, the solution

$$x_i = \frac{f_i(y_1, \dots, y_m)}{g_i(y_1, \dots, y_m)}$$

defines the coordinate functions of a rational map

$$\begin{aligned} \Phi: \mathbb{A}_{\mathbb{F}}^m &\dashrightarrow \mathbb{A}_{\mathbb{F}}^n \\ (a_1, \dots, a_m) &\mapsto \left(\frac{f_1(a_1, \dots, a_m)}{g_1(a_1, \dots, a_m)}, \dots, \frac{f_n(a_1, \dots, a_m)}{g_n(a_1, \dots, a_m)} \right) \end{aligned} \quad (1.7)$$

We will see in chapter 5 that applying the Kaltofen Yang algorithm to such a linear system restricts the domain of the rational map to the affine line $\Phi|_{\mathcal{L}_i}$. This implicitly restricts the coordinate function of the rational map, the solution of the linear system to the affine line $\mathcal{L}_i$ as well.

1.7.5 Modular black box for a parametric system of linear equations

Definition 1.7.1. Let $Ax = b$ be a parametric system of linear equations in m parameters $y_1, \dots, y_m$ with a non singular coefficient matrix $A \in \mathbb{Z}[y_1, \dots, y_m]^{n \times n}$, such that $\exists$ unique solution $x \in \mathbb{Z}[y_1, \dots, y_m]$ ie. $f \in \mathbb{F}[x_1, \dots, x_n]$. A modular black box B for $Ax = b$ is a program that on input $\alpha \in \mathbb{Z}^m$ and a prime p computes, $x(\alpha) = A^{-1}(\alpha)b(\alpha) \mod p$ (or $\frac{\det(A_i)}{\det(A)} \mod p$) ie. $B: \mathbb{Z}^m \rightarrow \mathbb{Z}_p^n$.

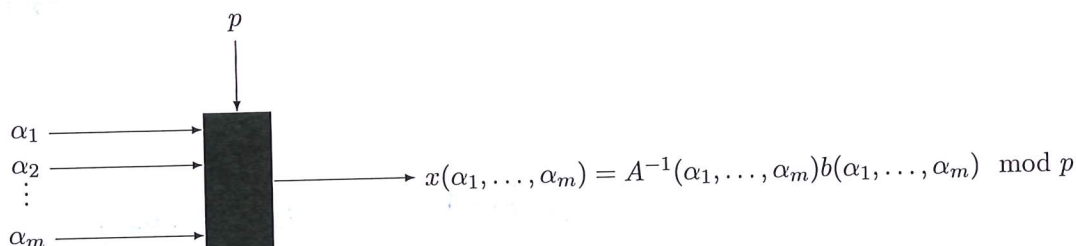
XX

X

missing α

$\mathbb{Z}(y_1, \dots, y_m)$

$$\frac{\det(A_i)(\alpha)}{\det(A)(\alpha)} \quad 1 \leq i \leq n$$



To avoid the expression swell caused in 1.6, we construct a modular black box for the parameterized linear system, evaluate it at randomly sampled points $\alpha \in \mathbb{Z}_p^m$, and find numerical solutions $x(\alpha) \in \mathbb{Z}_p^n$. This reformulates the problem of solving the parametric linear system to recovering a vector of multivariate rational functions from a numerical vector.

We then use Kaltofen Yang's sparse multivariate rational function interpolation to interpolate the sparse rational functions for each component $x_i = \frac{f_i(y_1, \dots, y_m)}{g_i(y_1, \dots, y_m)}$ of the solution vector x directly from the numerical solutions.

Kaltofen-Yang restricts the rational map of the linear system to an affine line

$$\bar{\Psi}_{\sigma^i, \beta} : \mathcal{L}_i \rightarrow \mathbb{A}_{\mathbb{Z}_p}^n$$

Evaluating the modular black box at $\mathcal{L}_i(x)$ gives us a vector of univariate rational functions.

$$\begin{aligned} \Phi|_{\mathcal{L}_i} : \mathcal{L}_i &\dashrightarrow \mathbb{A}_{\mathbb{Z}_p}^n \\ \Phi(\bar{\Psi}_{\sigma^i, \beta}(x)) &\mapsto \left(\frac{f_1(\bar{\Psi}_{\sigma^i, \beta}(x))}{g_1(\bar{\Psi}_{\sigma^i, \beta}(x))}, \dots, \frac{f_n(\bar{\Psi}_{\sigma^i, \beta}(x))}{g_n(\bar{\Psi}_{\sigma^i, \beta}(x))} \right) \in \mathbb{Z}_p(x)^n \end{aligned} \quad (1.8)$$

Thus, we can aggregate the numerical values component-wise across all the numerical solutions and use MQRFR to recover the univariate numerator and denominator of each component in the solution. We then proceed to apply Ben-Or Tiwari to recover the multivariate numerator and denominator. Finally, we use Maple's iratrecon to recover the rational pre-images in the end.

And Chinese remaindering?

```

A := Matrix([ [
                ] ], for m..20
b := Vector([
                ] );
X := [y1, y2];
B := proc(alpha :: list(integer), p :: prime)
    Y := { seq( x[i] = alpha[i], i = 1..nops(X) ) };
    Ab := Eval( <A|b>, Y ) mod p;

```

end;